



An open-source graphical framework for the design, simulation and evaluation of Optical Quantum Data Centers — circuit, network, compilation, scheduling and execution in one environment.

WHAT A RUN TELLS YOU

Five numbers that change when the network changes.

Entanglement fidelity

Quality of the pairs actually delivered to remote gates.

Job execution time (JET)

Wall-clock length of the distributed job, network included.

Communication overhead

Time and resources spent moving information, not computing.

Entanglement-resource utilization

How hard generation and memory resources are worked.

Penalty delay

Waiting caused by a remote gate outrunning its entangled pair.

SEE IT LIVE

ECOC 2026 · Demonstration D22

Tuesday 22 September 2026, 11:00–12:30, Pavilion 1 demo area. Drive it yourself: change the topology, the noise or the scheduling policy and watch the application-level numbers move.

RUNS ON

Windows · macOS · Linux
Standalone application,
no command line required.

CONTACT

Seyed Navid Elyasi
elyasi@chalmers.se
Chalmers University of Technology

DOWNLOAD

github.com/qnest-toolkit/qnest

Open source under the MIT License. Presented as a demonstration at the European Conference on Optical Communication (ECOC) 2026.

WACQT | Wallenberg Centre for Quantum Technology



Supported by the Swedish Research Council (VR) and the UK EPSRC Integrated Quantum Networks Hub (EP/Z533208/1).



Design the network. Compile the circuit. See what it costs.

A graphical toolkit for end-to-end simulation of optical quantum data centers.

ECOC 2026 · DEMO D22 · TUE 22 SEPT · PAVILION 1

CHALMERS × UNIVERSITY OF BRISTOL

OPEN SOURCE

THE GAP

The tools for quantum data center research don't talk to each other.

Circuit frameworks, distributed compilers and network execution environments each solve one layer well. Nothing connects them, so the question that matters most — how the optical network changes what the application actually achieves — falls between them.

Circuit frameworks

QISKIT · PYTKET · QASM

Strong circuit design and compilation, but a single-device view. The interconnect isn't part of the model.

Distributed compilers

PYTKET-DQC

Partition circuits across modules and allocate resources, but the communication they assume stays abstract — nothing ties the partition back to real fiber.

Network execution

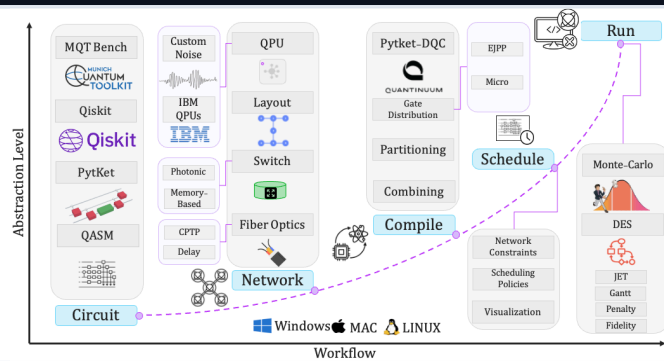
QOALA · NETSQUID

Rigorous node scheduling and protocol-level modelling of quantum communication, with no notion of the circuit that produced the traffic.

QNEST

THE CONNECTIVE TISSUE

Rather than reimplementing any layer, QNEST wires these tools into one workflow and adds what was missing: optical network design, switching protocols, and a graphical environment over the whole run.



WORKFLOW AND SOFTWARE ARCHITECTURE

THE WORKFLOW

Five stages, in order, end to end.

Each stage hands its output to the next one.

01 Circuit · MQT Bench

Choose the benchmark circuit and the number of qubits for the run, and set the duration of every gate and process.

02 Network

Define the topology and the connectivity between QPUs: beam splitters, switches, quantum memories and optical links, with qubit counts, T1 and T2, gate and readout errors.

03 Distribute · pyket-dqc

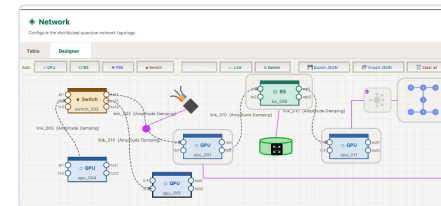
pyket-dqc compiles the circuit against that network and distributes it over the QPUs, exposing remote-gate dependencies and entanglement demand.

04 Schedule · Qoala + NetSquid

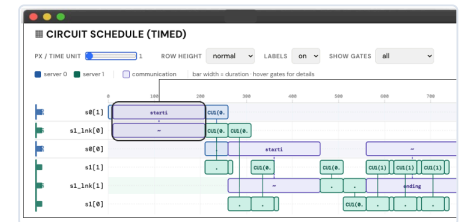
The distributed circuit is passed to the QPUs and scheduled by strategies defined in Qoala and NetSquid, then drawn as an interactive Gantt chart exported as standalone HTML.

05 Run · switching protocols

Our switching protocols are applied for either an all-photonic or a memory-assisted architecture, and results come back as detailed tables, down to the individual event.



NETWORK DESIGNER



CIRCUIT SCHEDULE

BUILT ON

MQT Bench for workloads · pyket-dqc for distribution · Qoala and NetSquid for scheduling and execution, both from TU Delft. Circuit scheduling and entanglement generation no longer rest on in-house heuristics.